

# ID1063 — Introduction to Programming in C

Consolidated exam-prep summary — Weeks 1–7 (Basics through Functions)

Compiled from lecture slides for full-marks revision. Assumes existing knowledge of C; focused on exact C semantics, common trick points, and worked examples the instructor emphasized.

---

## How to use this document

- Sections follow the course's topic order (Weeks 1–7), not verbatim slide order.
- **GOTCHA** boxes (orange) flag the exact traps the instructor pointed out in class — these are prime territory for negative-marking MCQs.
- **Q/A** boxes (green) are worked slide questions with the reasoning made explicit — trace through these instead of just reading the answer.
- Code is written exactly as shown in the slides (including the instructor's variable naming and style) so it matches what you've seen in lecture/lab.

## Exam-relevant facts from Week 7 logistics

| Assessment                   | Date/Window      | Weightage |
|------------------------------|------------------|-----------|
| Midsem theory                | Wed, 16 Sep 2026 | 22.5%     |
| Midsem lab / Lab Exam 1      | Sat, 19 Sep 2026 | 20%       |
| Endsem theory                | Wed, 11 Nov 2026 | 27.5%     |
| Endsem lab                   | Sat, 14 Nov 2026 | 25%       |
| Lab attendance + submissions | Continual        | 5%        |

Theory exams are multiple choice **with negative marking** — the GOTCHA boxes throughout this document target exactly the kind of subtle-semantics questions that show up in such exams.

# 1. Program Structure, Compilation & Variables (Week 1)

## 1.1 From source code to a running program

- Workflow: Understand problem -> plan -> write C source code -> compile -> run.
- **Compiler** translates C source into an executable; **gcc** = GNU C Compiler.
- Steps: (1) write `hello.c` (plain text file), (2) compile: `gcc hello.c -o hello`, (3) run: `./hello`.
- If gcc reports an error, fix the *first* reported error and recompile — later errors are often just consequences of the first.

## 1.2 First C program & structure

```
// My first C program
#include <stdio.h>
int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

- `#include <stdio.h>` brings in standard input/output declarations (needed for `printf/scanf`).
- `main` is the special function the OS calls first when the program runs.
- `return 0;` from `main` tells the OS the program completed successfully.
- Comments: `// single line`. `/* multi-line */`: comments are ignored by the compiler.

**GOTCHA — Errors:** Two categories: **syntax errors** (compiler catches these; code won't compile) vs **logical errors** (code compiles and runs, but produces the wrong answer).

## 1.3 Functions (first look)

- A function is a named, grouped block of statements that (usually) produces an output — like a mathematical function  $f(x) = x + 42$ .
- `main` is always executed first automatically.
- In the first weeks programs contain only `main`; user-defined functions arrive in Week 7.

## 1.4 Variables & basic types

- A variable is a named storage location in memory; it has a **type** (fixes size and interpretation of stored bits) and a **name**.
- Declaration: `int x;` | Assignment: `x = 10;` | Initialization (both together): `int x = 10;`
- A variable declared but never initialized is **uninitialized** (contains garbage — do not assume 0).

| Type   | Typical size                                 | Typical range / precision                                             |
|--------|----------------------------------------------|-----------------------------------------------------------------------|
| char   | 1 byte (8 bits)                              | signed: -128 to 127; unsigned: 0 to 255                               |
| int    | 4 bytes                                      | -2,147,483,648 to 2,147,483,647                                       |
| float  | 4 bytes                                      | $\approx \pm 3.4 \times 10^{38}$ ; ~6–7 significant decimal digits    |
| double | 8 bytes (1 sign + 11 exponent + 52 mantissa) | $\approx \pm 1.8 \times 10^{308}$ ; ~15–16 significant decimal digits |

**GOTCHA — printf placeholders:** `printf` uses `%f` for **both** float and double — but `scanf` needs `%f` for float and `%lf` for double. Mixing these up in `scanf` is a classic bug.

## 2. Input/Output, Number Representation & Expressions (Week 2)

### 2.1 printf format specifiers

| C type | printf | scanf | Notes                   |
|--------|--------|-------|-------------------------|
| int    | %d     | %d    | whole numbers           |
| char   | %c     | %c    | one character           |
| float  | %f     | %f    | ~7 digits precision     |
| double | %f     | %lf   | ~15–16 digits precision |

```
int year = 2026;
printf("This course begins in %d.\n", year);    // This course begins in 2026.
```

### 2.2 scanf — reading input

```
int length;
scanf("%d", &length);           // & gives scanf the ADDRESS of length
printf("You entered %d\n", length);
```

**GOTCHA — Missing &:** Forgetting & before the variable in scanf (e.g. `scanf("%d", length);`) is one of the most common starter bugs — scanf needs the memory *address* to write into, not the value.

```
int length, width, height;
scanf("%d %d %d", &length, &width, &height);    // space-separated: reads 3 ints in one call
```

### 2.3 Output buffering

- printf output can sit in a buffer instead of appearing immediately — so a prompt printed just before scanf may not show up before the program waits for input (behaviour can vary across systems).
- Fix (whole program): `setvbuf(stdout, NULL, _IONBF, 0);` at the start of main.
- Fix (per statement): `fflush(stdout);` right after the printf.
- Stream picture: keyboard/input file -> stdin -> your program -> stdout -> terminal/output file.

**GOTCHA — Reading a char after reading a number:** After `scanf("%d", &num);`, a leftover newline sits in the input buffer. A following `scanf("%c", &grade);` would read that leftover newline instead of your typed character. Fix: put a leading space — `scanf(" %c", &grade);` — the space skips leading whitespace including newlines.

### 2.4 Number representation

- Integers stored in **binary** (base 2). Hexadecimal is base 16 (digits 0-9,A-F), usually written with a 0x prefix.
- **Negative integers** use **two's complement**: flip all bits of the positive value, then add 1. E.g. -5: 5 = 00000101 -> flip -> 11111010 -> +1 -> 11111011. This lets the CPU use the same circuit for addition and subtraction.
- **Characters** are stored as 1-byte integers (ASCII). 'A' = 65, 'a' = 97. `printf("%c")` looks up the integer in the ASCII table to print the symbol — this is why `char c = 'a'; c+1` gives 'b' when printed as a char.
- **double** is the default/native floating type in C (use it unless memory-constrained); **float** saves space (4 bytes) but has less precision. Watch out for floating-point precision errors.

### 2.5 Arithmetic expressions & precedence

- Like BODMAS: parentheses first, then \* / %, then + -. Same-level operators evaluate left to right.
- `int a = 2 + 3 * 4; -> 14. int b = (2 + 3) * 4; -> 20.`
- % is remainder/modulo: `17 % 5 = 2. 17 / 5 = 3` (integer division truncates).

**Q:** `int a = 25, b = 10; What do x = 2*a/b and y = a/b*2 give?`

**A:** `x = (2*25)/10 = 50/10 = 5. y = (25/10)*2 = 2*2 = 4.` Same numbers/operators, different left-to-right grouping => different results.

**GOTCHA — Integer division truncation:** `double average1 = (10 + 11) / 2;` gives **10.0**, not 10.5 — the division `(10+11)/2` happens entirely in **int** arithmetic first (`21/2 = 10`), and the fraction is already lost before it's stored

in the double. Use 2.0 or a cast to force floating-point division: (double)total / count.

```
int total = 21, count = 2;  
double average = (double) total / count;    // cast happens first -> 10.5
```

## 2.6 Relational & logical operators

| Relational | Meaning               | Logical | Meaning |
|------------|-----------------------|---------|---------|
| <          | less than             | &&      | AND     |
| <=         | less than or equal    |         | OR      |
| >          | greater than          | !       | NOT     |
| >=         | greater than or equal |         |         |
| ==         | equal to              |         |         |
| !=         | not equal to          |         |         |

- In C: 0 means False, any nonzero value means True. A comparison itself evaluates to 1 (true) or 0 (false).  
#include <stdbool.h> gives a bool type for clarity.
- Precedence (high->low): ! (right-to-left) > && (left-to-right) > || (left-to-right). Use parentheses to avoid confusion.

## 3. Conditionals (Week 3)

### 3.1 if / if-else

```
if (condition) {
    statements to run if condition is true
} else {
    statements to run otherwise
}
```

- The condition is evaluated **once**. Exactly one of the two blocks runs, then execution continues after the whole if-else.

### 3.2 else-if chains

```
if (marks >= 90)    printf("A\n");
else if (marks >= 80) printf("B\n");
else if (marks >= 70) printf("C\n");
else if (marks >= 60) printf("D\n");
else                printf("F\n");
```

- Conditions are tested top to bottom; as soon as one is true its block runs and **every later branch in that chain is skipped** — at most one branch executes.
- **Order matters**: put the most restrictive/highest threshold first. Once `marks >= 90` fails, the next test already "knows" `marks < 90`, so `marks >= 80` alone is enough for grade B.
- Add a validity guard first when input may be out of range: `if (marks < 0 || marks > 100) ... invalid` before the grade checks.

### 3.3 Nested conditionals

- A conditional can contain another; the outer condition picks a broad case, the inner one refines it. Use nesting when the second question only makes sense given the first answer. Indent to show the structure.

```
if (age < 12) {
    price = 8;
} else {
    price = 15;
    if (is_student) { price = 10; }
}
```

### 3.4 Short-circuit evaluation (very testable)

- `A && B`: if A is false, B is **not evaluated**.
- `A || B`: if A is true, B is **not evaluated**.
- Evaluated strictly left to right — this can be used to guard against invalid operations, but **only if the protective test comes first**.

```
// SAFE: denominator!=0 checked first, short-circuits before the division
if (denominator != 0 && numerator / denominator > 2) { ... }
```

```
// UNSAFE: division may run before the zero-check
if (numerator / denominator > 2 && denominator != 0) { ... }
```

### 3.5 Common conditional bugs (high-yield for MCOs)

**GOTCHA — Chained range comparisons don't work like math**: `if (0 <= marks <= 100)` does NOT test a range in C. It evaluates left to right: `0 <= marks` first, giving 0 or 1; that result (0 or 1) is then compared with 100, which is **always true**. Correct form: `0 <= marks && marks <= 100`.

**GOTCHA — Assignment (=) vs equality (==)**: `if (x = 0)` **assigns** 0 to x (and the condition becomes false, since the assigned value 0 is used as the condition) — it does not compare. After this line, x is now 0 regardless of its old value. Use `==` to compare.

**GOTCHA — Stray semicolon after if**: `if (temperature > 40);` — the semicolon immediately ends the if statement with an **empty** body. The following `{ printf(...); }` block then runs unconditionally, every time, regardless of the condition.

**GOTCHA — Dangling else:** An else always binds to the **nearest unmatched if**, not necessarily the one that looks (by indentation) like its match. `if (logged_in) if (is_admin) ... else ...` — the else belongs to `if (is_admin)`, not `if (logged_in)`, even if indentation suggests otherwise. Use braces to make intent explicit.

**Q:** For `if (a > b) print a; else if (b+a > 10) print sum-msg; else print equal-msg;` — when is the third branch printed?

**A:** Only when both earlier conditions are false:  $a \leq b$  AND  $a+b \leq 10$ .

## 4. Loops: while, do-while, for (Week 4 recap + Week 6)

### 4.1 The three parts every loop needs

- Every loop needs: an **initial value**, a **condition** (termination test), and an **update**.

### 4.2 while — checks condition BEFORE each iteration

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    i++;
}
```

- If the condition is false the first time, the body never runs (0 iterations possible).
- Prefer while when the number of repetitions is **not known in advance** — e.g. reading until a sentinel value like -1 stops input.

```
int reading, sum = 0;
scanf("%d", &reading);
while (reading != -1) {           // -1 is a sentinel value
    sum = sum + reading;
    scanf("%d", &reading);
}
```

### 4.3 do-while — body runs AT LEAST ONCE

```
int row = 1;
do {
    printf("* ");
    row = row + 1;
} while (row <= numrows);
```

- Condition is checked *after* the body, so the body always executes at least once — appropriate only when you're sure at least one iteration should happen.

### 4.4 for — compact counting loop

```
for (initialization; condition; update) {
    statements;
}
for (int i = 0; i < 5; i = i + 1) {
    printf("%d ", i);
}
// Execution order: initialize once -> test -> body -> update -> test again -> ...
```

- Bundles initial value/condition/update together — use it when those three clearly belong together (typically indexing).
- A while and an equivalent for visiting indices 0..n-1 do the same work; choose based on whether the iteration count is known ahead of time.

### 4.5 break and continue

- break and continue affect the **nearest enclosing loop** only.
- continue skips the rest of the current iteration's body and jumps to the update/condition check (careful: in a while loop, a continue placed before the increment can create an infinite loop, since the update step gets skipped too).

### 4.6 Nested loops

- Outer loop picks the current "row"; inner loop completes all work for that one outer iteration; then outer moves on.
- **Classic bug: the inner loop's counter must be reset before every new outer iteration** — otherwise the inner loop only fully executes on the first outer pass and does nothing (or partial work) after that, since its counter is already past the limit.

```
// BUGGY: j is never reset, so after i's first pass, j stays >= numcols forever
int i = 1, j = 1;
while (i <= numrows) {
    while (j <= numcols) { printf("* "); j = j + 1; }
}
```

```

    printf("\n");
    i = i + 1;
}

// FIXED: reset j at the start of every outer iteration
int i = 0, j;
while (i < numRows) {
    j = 0;
    while (j < numcols) { printf("* "); j = j + 1; }
    printf("\n");
    i = i + 1;
}

```

**GOTCHA — Scope of a loop-declared variable:** If the inner counter is declared *inside* the outer loop's body (e.g. `int j = 0;` as the first line inside the outer `while`), it is naturally re-created fresh every outer iteration — but it also then does **not exist** outside the outer loop; using it after the loop is a compile error.

#### 4.7 Boundary cases — always check these

- For a loop: check 0 iterations, 1 iteration, and the last valid index.
- For array-filling loops, an off-by-one in the condition (`i <= n` instead of `i < n`) writes one element past the end of the array — C does not reliably stop this out-of-bounds access, so it may silently corrupt other data instead of crashing.

**Q:** `for (int i = 2; i <= 8; i = i + 3) printf("%d ", i);` — what prints?

**A:** Trace `i = 2, 5, 8`, (11 fails the test). Output: `2 5 8`.

**Q:** `int a[4]={3,8,2,7}; int sum=0; for(i=0;i<4;i=i+2) sum+=a[i];` — what prints?

**A:** Only indices 0 and 2 are visited: `a[0]=3, a[2]=2, sum = 5`.

## 5. Arrays (Week 5)

### 5.1 Declaring & indexing

```
int marks[5]; // capacity 5, uninitialized
int marks[5] = {78, 85, 92, 64, 88}; // capacity 5, initialized
```

- An array stores several values of the **same type** under one name; each has a numbered **index** starting at 0.
- For capacity  $n$ , valid indices are  $0 \dots n-1$ . `marks[5]` for a capacity-5 array is **invalid** (out of bounds).

**GOTCHA — Out-of-bounds access is not caught:** C does not reliably stop an out-of-bounds array access at run time — the program may read/write memory belonging to something else, causing unpredictable behaviour instead of a clean error.

### 5.2 Reading and processing arrays

```
int values[50], n, i = 0;
scanf("%d", &n); // precondition: 0 <= n <= 50
while (i < n) {
    scanf("%d", &values[i]);
    i = i + 1;
}
```

- Pattern: start  $i$  at 0, use `values[i]`, update a running summary (sum/count/max), increment  $i$ , stop when  $i$  reaches  $n$ . Condition  $i < n$  exactly covers valid indices  $0..n-1$ .

### 5.3 Finding max and its index

```
int maximum = marks[0];
int max_index = 0;
for (int i = 1; i < n; i = i + 1) {
    if (values[i] > maximum) {
        maximum = values[i];
        max_index = i;
    }
}
```

- Note: uses **strictly greater than**, so on ties the *first* occurrence of the maximum is kept as `max_index` (later equal values don't overwrite it).

## 6. Array Algorithms & Strings (Week 6)

### 6.1 Character arrays and the null terminator

- A **string** in C is just a character array with a special terminating character `'\0'` (the "null character") marking the end.
- `'\0'` is not the character `'0'` or a space — it's the byte value 0.
- The array can be larger than the string it holds; the string simply ends at the first `'\0'`.

```
char word[4];
word[0]='c'; word[1]='a'; word[2]='t'; word[3]='\0';
```

### 6.2 Initializing strings (3 ways) — and one illegal way

```
char str[] = "Hello";           // size auto = 6 ('H','e','l','l','o','\0')
char str[10] = "Hello";         // first 5 chars set, rest auto-filled with '\0'
char str[] = {'H','e','l','l','o','\0'};
```

```
char str[10];
str = "Hello";                  // ERROR: array types are not assignable
```

**GOTCHA — Arrays cannot be assigned after declaration:** Just like integer arrays, you cannot write `str = "Hello"`; once `str` is already declared as an array — this does not compile. Assignment to a whole array is only possible combined with initialization at declaration time.

### 6.3 Reading strings safely

```
char word[20];
scanf("%19s", word);           // no & before word! reads ONE word (stops at whitespace)
printf("%s\n", word);
```

```
char str[50];
scanf("%49[^\n]", str);        // reads up to 49 chars, stops at newline (keeps spaces)
```

- Do not use & before the array name in scanf for strings** (unlike `int/char/float`) — reason becomes clear with pointers (Week 8+); the array name already acts as the address.
- `%s` reads only up to the first whitespace — "Hello World" input gives just "Hello".
- Always size the format specifier one less than the array size (e.g. `%19s` for a 20-char array) to reserve room for the `'\0'` — buffer-overflow protection.

### 6.4 String length and traversal

```
int length = 0;
while (word[length] != '\0') {
    length = length + 1;
}
// Stop at '\0', NOT at the array's declared capacity -- the string may use fewer slots.
```

- Common string algorithms follow the same shape: loop with condition `word[i] != '\0'` (vowel counting, palindrome check via `str[i]==str[len-1-i]`, longest run of one repeated character, etc.).

### 6.5 string.h library functions

| Function                           | Purpose                                                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>strlen(str)</code>           | Length of string, excluding <code>'\0'</code> .                                                                          |
| <code>strcpy(dest, src)</code>     | Copies <code>src</code> into <code>dest</code> (incl. <code>'\0'</code> ). Overflow risk if <code>dest</code> too small. |
| <code>strncpy(dest, src, n)</code> | Copies up to <code>n</code> chars. Safer alternative to <code>strcpy</code> .                                            |
| <code>strcat(dest, src)</code>     | Appends <code>src</code> to end of <code>dest</code> . <code>dest</code> must have room for the result.                  |
| <code>strncat(dest, src, n)</code> | Appends up to <code>n</code> chars. Safer alternative to <code>strcat</code> .                                           |
| <code>strcmp(s1, s2)</code>        | Lexicographic compare: 0 if equal, <0 if <code>s1&lt;s2</code> , >0 if <code>s1&gt;s2</code> .                           |
| <code>strncmp(s1, s2, n)</code>    | Compares only the first <code>n</code> characters.                                                                       |

## 6.6 Common string mistakes

- `char word[20]` holds at most **19** typed characters — one slot must be reserved for `'\0'`.
- `%s` reads one word, not a full line with spaces (use `%[^\n]` for that).
- Use `%c` for a single character, `%s` for a string.

## 6.7 Good coding practice reminders from slides

- Indent consistently: move one level right after `{`, one level left before `}` (4 spaces per level).
- One loop, one purpose: initialize before the loop, update inside the relevant condition, keep the loop variable only as an index.
- Trace a small input by hand before running; test 0 iterations, 1 iteration, and the last valid index; for array algorithms also test `n = 1` and repeated values.
- Compile and fix the *first* error before changing anything else.

## 7. Functions (Week 7)

### 7.1 Why functions

- Functions are named, reusable blocks of code for a specific task — avoid repetition, improve readability (main reads like plain English), improve maintainability (fix behaviour in one place).
- Familiar functions like `printf/scanf` are pre-written in the `stdio` library. `main` itself is a special function invoked by the OS.

### 7.2 Anatomy of a function

```
return_type name ( parameters/arguments ) {  
    // body: statements that do the job  
}
```

```
int square(int value) {  
    return value * value;  
}
```

- **return type**: type of value the function gives back (use `void` if it returns nothing).
- **parameter** = variable named in the *definition*; **argument** = the actual value/expression supplied in a *call*. E.g. definition `int maximum(int first, int second)` vs call `maximum(scoreA, scoreB)`.

```
void showWelcome(void) {           // void = no value used/returned  
    printf("Welcome to ID1063!\n");  
}  
  
int main(void) {  
    showWelcome();  
    return 0;  
}
```

### 7.3 Execution order of a function call

- 1) `main` reaches the call. 2) control jumps into the called function. 3) the function body runs (computing/storing a return value if any). 4) control returns to `main` and continues from the next line after the call.

**GOTCHA — return is NOT printf**: `return value*value;` sends a result back to the **caller** — it does not display anything. `printf` displays characters on screen but does not store anything for later use. A printed result is not automatically saved anywhere.

### 7.4 Local variables and scope

```
int square(int value) {  
    int ans = value * value;  
    return ans;  
}  
// 'ans' does NOT exist in main -- it is local to square().
```

- A variable declared inside a function is **local**: it is created when the function starts and destroyed when it returns; it is not visible to (or from) other functions, including `main`.

### 7.5 Pass-by-value (critical concept)

```
void addOne(int number) {  
    number = number + 1;           // modifies only the LOCAL COPY  
}  
  
int main(void) {  
    int count = 4;  
    addOne(count);  
    printf("Count value is: %d\n", count);    // prints 4, NOT 5  
    return 0;  
}
```

**GOTCHA — Ordinary parameters are copies**: C passes ordinary (non-pointer) arguments **by value**: the function receives a *copy*. Changes made inside the function to the parameter **never** affect the caller's original variable. (Pointers, covered in a later week, are the way to let a function modify the caller's variable.)

## 8. Quick-Fire Gotcha Checklist (review right before the exam)

- printf uses %f for float AND double; scanf needs %f for float but %lf for double.
- Missing & before a scanf variable (except for whole strings/char-arrays with %s).
- Output buffering: a prompt may not appear before scanf unless flushed (setvbuf/fflush).
- A leftover newline after scanf("%d",...) can be consumed by a following scanf("%c",...) unless you use " %c" (leading space).
- Two's complement: negative n = flip bits of n, add 1.
- char is just a 1-byte integer under the hood; arithmetic on chars uses ASCII codes.
- Integer division truncates:  $(10+11)/2 = 10$ , not 10.5 -- cast one operand to double, or divide by 2.0, BEFORE the division happens.
- Operator precedence:  $() > * / \% > + -$ ; same level = left to right.
- $0 \leq x \leq 100$  is NOT a range check in C -- it's  $(0 \leq x)$  compared with 100. Use &&.
- if (x = 0) assigns, it does not compare. Use ==.
- if (cond); { ... } -- stray semicolon makes the block run unconditionally.
- else binds to the nearest unmatched if, regardless of indentation.
- && and || short-circuit left to right -- put the safety/guard condition FIRST.
- Off-by-one in loop bounds ( $i \leq n$  instead of  $i < n$ ) writes/reads one past a valid array index; C will not stop you.
- Nested loops: reset the inner loop's counter at the start of every outer iteration.
- Arrays cannot be assigned with '=' after declaration (str = "Hello"; does not compile).
- %s in scanf stops at the first whitespace; use %[^\n] to read a full line.
- A string's logical end is the first '\0', which may come before the array's declared capacity.
- Reserve one extra array slot for '\0' when sizing string buffers/format specifiers.
- Function parameters are passed by value in C: a function cannot change the caller's original variable through an ordinary parameter.
- A variable local to a function does not exist outside it (undeclared-identifier compile error if you try).